

Duet

A Dual-Branch Alternating Sponge Hash Function

Submitters:

Yaobin Shen, Xiamen University

`yaobin.shen@xmu.edu.cn`

Tingting Cui, Hangzhou Dianzi University

`cuitingting@hdu.edu.cn`

Ziyang Luo, Xiamen University

`ziyangluo@stu.xmu.edu.cn`

Yan He, Hangzhou Dianzi University

`yan_he@hdu.edu.cn`

Feng Li, Xiamen University

`lifeng8425@stu.xmu.edu.cn`

Corresponding submitter:

Tingting Cui, Hangzhou Dianzi University

`cuitingting@hdu.edu.cn`

June 30, 2026

Contents

1	Introduction	2
2	Algorithm Description	2
2.1	Notation and Parameters	3
2.2	Overall Structure and State Partitioning	4
2.3	Message Padding	5
2.4	Absorbing Phase	6
2.5	Squeezing Phase and Output Mechanism	6
2.6	Specification of the Permutation f	7
3	Design Rationale	10
3.1	Shared-Capacity Alternation and Capacity Feed-Forward	11
3.2	Three-dimensional Permutation f	11
3.3	Parameter Selection and Security Targets	12
4	Security Claims and Analysis	13
4.1	Security Claims	13
4.2	Theoretical Security	13
4.2.1	Model and Assumptions	13
4.2.2	Indifferentiability Background of the Alternating Structure	14
4.2.3	Classical Advantage Bounds	14
4.2.4	Quantum Generic Bounds	16
4.2.5	Additional Structural Security Considerations	16
4.3	Concrete Security	16
4.4	Security of Permutation f	17
4.4.1	Security of the S-box	17
4.4.2	Security against Differential Attacks	19
4.5	Statistical Randomness Test Results	19
5	Performance Evaluation	20
5.1	Performance analysis	20
5.2	Software Implementation	20
5.2.1	Benchmark Setup	20
5.2.2	Performance Results and Resource Consumption	21
5.3	Hardware Implementation	24
6	Features	24
6.1	Innovative Aspects of the Algorithm	24
6.2	Simplicity and Structural Uniformity	25
6.3	Flexibility and Parameter Scalability	25
6.4	Implementation Advantages	25

1 Introduction

As cryptographic applications move toward higher security levels and resistance against quantum adversaries, cryptographic hash functions are expected to provide reliable guarantees against both classical and quantum attacks. In response to this requirement, we propose Duet, a permutation-based cryptographic hash function built on dual-branch alternating updates and capacity feed-forward. As suggested by its name, Duet absorbs messages through two coordinated state branches, while a shared capacity state serves as the central coupling component throughout the computation.

The core design of Duet is a dual-branch alternating sponge structure with capacity feed-forward. The internal state consists of two rate branches and one shared capacity branch. For each message block, Duet performs two permutation calls in sequence: the first updates one rate branch and the capacity state, while the second updates the other rate branch using a linearly transformed message block and the capacity value produced by the first stage. Thus, the two branches are not independent parallel computations, but are serially coupled through the shared capacity chain. This alternating schedule is motivated by the Alternating Sponge framework, whose ideal-permutation analysis provides indistinguishability security beyond the birthday bound associated with the capacity size, and serves as the structural security background of Duet. The capacity update further applies a feed-forward XOR rule, combining the input capacity value with the capacity part of the permutation output.

Duet provides three fixed-digest instances based on two permutation widths. Duet-512 uses a 960-bit permutation with $r = 384$ and $c = 576$, targeting a 512-bit digest. Duet-768 and Duet-1024 both use a 1920-bit permutation, but with different rate-capacity partitions. Specifically, Duet-768 uses $r = 1088$ and $c = 832$, targeting a 768-bit digest, while Duet-1024 uses $r = 768$ and $c = 1152$, targeting a 1024-bit digest. Beyond these fixed-digest instances, the same squeezing mechanism also supports capacity-based extendable output for application-dependent output lengths.

For each supported fixed digest length $h \in \{512, 768, 1024\}$, Duet targets $\frac{h}{2}$ -bit collision security, h -bit preimage security, and h -bit second-preimage security in the classical setting. In the quantum setting, its target complexities match the generic bounds for a random h -bit hash function, namely about $2^{\frac{h}{3}}$ queries for collision attacks and about $2^{\frac{h}{2}}$ queries for preimage and second-preimage attacks. Since the digest is derived from the capacity branch and the complete internal state is not directly exposed, standard Merkle–Damgård length-extension attacks are not applicable. The same core construction can also be used to support XOF, MAC, and KDF functionalities through appropriate encoding and domain separation.

The remainder of this document is organized as follows. Section 2 specifies the Duet algorithm, including its padding, absorbing, squeezing, and underlying permutations. Section 3 explains the design rationale and parameter selection. Section 4 presents the security claims and analysis under classical and quantum attack models. Section 5 reports software and hardware implementation performance. Section 6 summarizes the main features of Duet. The accompanying materials include reference implementations, optimized implementations, and test vectors for reproducibility and public evaluation.

2 Algorithm Description

This section presents the full specification of the Duet cryptographic hash algorithm. Duet is a permutation-based iterative hash function with an extendable-output interface [1, 2]. It is built from two independently invoked b -bit permutations, denoted by f_1 and f_2 , where $b = r + c$. The state is organized into two r -bit rate branches and one shared c -bit capacity branch. A two-stage serial alternating update couples the two rate branches through the shared capacity chain.

A distinctive feature of Duet is the use of capacity feed-forward, motivated by previous feed-

forward sponge designs [5, 6, 7], with capacity-based squeezing. After each permutation call, the capacity part of the permutation input is fed forward to the capacity part of the permutation output. During squeezing, Duet extracts output from the capacity branch rather than from the rate branches. With a squeezing rate $r' \leq c$, the construction supports outputs of variable length; in particular, an h -bit digest is obtained by taking the first h output bits under the condition $h \leq c$. Under the assumptions analyzed in Section 4.2, Duet targets $\frac{h}{2}$ -bit collision security, h -bit preimage security, and h -bit second-preimage security in the classical setting for the supported maximum message length and selected parameters. The selected parameters balance state size, security margins, and implementation performance.

2.1 Notation and Parameters

Before describing the algorithm, we define the notation and parameters used throughout the specification.

Parameters and inputs.

- r : the absorption rate, i.e., the bit-length of each message block absorbed per iteration.
- c : the capacity size, used in the security bounds of the construction.
- b : the width of each underlying permutation, where $b = r + c$.
- h : the requested digest length in bits.
- r' : the squeezing rate, i.e., the number of bits extracted in each squeezing step. For a selected Duet- h instance, r' is not an external input but is fixed as $r' = h \leq c$.
- ν : the requested total output length in bits, which may be larger than r' .
- M : the input message of arbitrary bit-length.
- M_i : the i -th r -bit message block after padding.
- z_i : the i -th output fragment generated during squeezing.

State components.

- $A \in \{0, 1\}^r$: the upper rate branch.
- $B \in \{0, 1\}^r$: the middle rate branch.
- $C \in \{0, 1\}^c$: the shared capacity branch.
- $f_1, f_2 : \{0, 1\}^{r+c} \rightarrow \{0, 1\}^{r+c}$: two independently invoked underlying permutations.
- f : the unified representation of underlying permutations f_1 and f_2 , i.e. $f \in \{f_1, f_2\}$.
- $\sigma(\mathbb{F}_2^r \rightarrow \mathbb{F}_2^r)$: a linear permutation applied to the message block before the second-stage permutation. For the supported digest sizes, σ is a fixed permutation of 64-bit words. For example, in the 512-bit digest version ($r = 384$), if $M_i = W_0 \parallel W_1 \parallel W_2 \parallel W_3 \parallel W_4 \parallel W_5$, then σ uses the word order $(3, 0, 5, 1, 4, 2)$, that is,

$$\sigma(M_i) = W_3 \parallel W_0 \parallel W_5 \parallel W_1 \parallel W_4 \parallel W_2.$$

In the 1024-bit digest version ($r = 768$), σ is defined similarly as a fixed permutation of 12 words, with order $(8, 2, 11, 0, 5, 9, 3, 10, 1, 7, 4, 6)$. In the 768-bit digest version ($r = 1088$), σ is defined as a fixed permutation with order $(9, 2, 15, 0, 6, 12, 4, 16, 1, 10, 7, 14, 3, 11, 5, 13, 8)$.

Operation symbols:

- $\oplus$: bitwise exclusive-or.
- $\parallel$: concatenation of bit strings.
- $|X|$: the bit length of a bit string X .
- $X[a : b]$: the substring of X from index a to index b , inclusive.
- $\text{trunc}_{r'}(X)$: the first r' bits of the bit string X .

Capacity feed-forward call: For $s \in \{1, 2\}$, $R \in \{0, 1\}^r$, and $C \in \{0, 1\}^c$ define

$$F_s(R, C) = (R', C')$$

by first computing

$$(R', D) = f_s(R \parallel C),$$

and then applying the feed-forward update

$$C' = D \oplus C.$$

Thus the capacity output of the underlying permutation is not used directly; it is XORed with the capacity input before becoming the next capacity value.

2.2 Overall Structure and State Partitioning

Duet uses a dual-branch state with a shared capacity chain. Its persistent chaining state consists of two r -bit rate branches A and B , and one c -bit capacity branch C . Thus, the persistent state has size $2r + c$ bits. The feed-forward XOR may require an additional c -bit temporary buffer to keep the input capacity value during a permutation update. This buffer is temporary and does not form an additional persistent state branch. Each underlying permutation call operates on one rate branch together with the shared capacity branch, so the width of each permutation remains $b = r + c$.

The algorithm evolves the state through two serial stages. During absorption, the message block is injected into the upper branch before the first-stage permutation f_1 , while a linearly transformed copy of the message block is injected into the middle branch before the second-stage permutation f_2 . During squeezing, the construction no longer outputs from the rate branches; instead, output fragments are extracted from the capacity branch. After every permutation call, the capacity output is XORed with the corresponding capacity input, forming the capacity feed-forward update.

Figure 1 illustrates the overall data flow of Duet, including the two-stage absorbing process, the capacity feed-forward path, and the capacity-based squeezing phase.

The complete operation of Duet is summarized in Algorithm 1. The detailed padding, absorbing, and squeezing procedures are described in the following subsections.

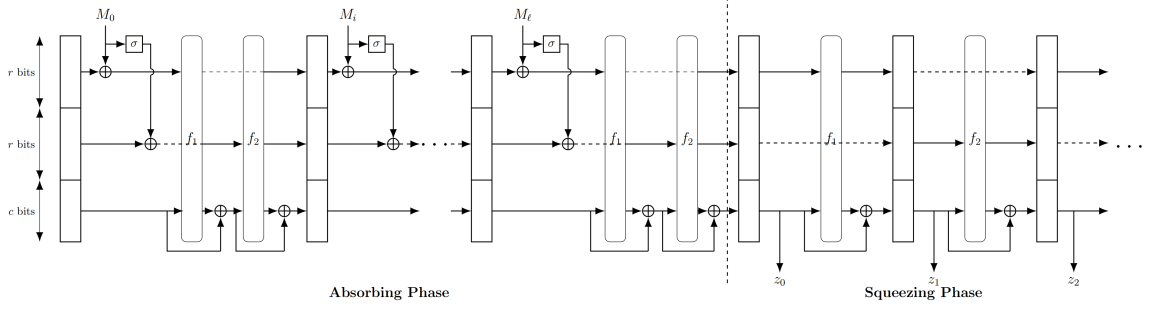


Figure 1: Overall structure of Duet with capacity feed-forward

Algorithm 1 Duet Operation Process

Require: Message M , desired output length $\nu \geq 1$, selected Duet- h instance with parameters h, r, c

Ensure: Output string $Z[0 : \nu - 1]$

```

1:  $r' \leftarrow h$ 
2: // Padding
3:  $M^* \leftarrow \text{Pad}_{10^*}(M, r)$ 
4: // Parse  $M^*$  into  $r$ -bit blocks:
5:  $M^* = M_0 \parallel M_1 \parallel \dots \parallel M_\ell$ , where  $M_i \in \{0, 1\}^r$  for  $0 \leq i \leq \ell$ .
6: // State Initialization
7:  $A \leftarrow 0^r$ ;  $B \leftarrow 0^r$ ;  $C \leftarrow 0^c$ 
8: // Absorbing Phase
9: for  $i = 0$  to  $\ell$  do
10:    $(A, D_1) \leftarrow f_1((A \oplus M_i) \parallel C)$ 
11:    $C \leftarrow D_1 \oplus C$ 
12:    $(B, D_2) \leftarrow f_2((B \oplus \sigma(M_i)) \parallel C)$ 
13:    $C \leftarrow D_2 \oplus C$ 
14: end for
15: // Squeezing Phase
16:  $Z \leftarrow \text{trunc}_{r'}(C)$ 
17: while  $|Z| < \nu$  do
18:    $(A, C) \leftarrow F_1(A, C)$ 
19:    $Z \leftarrow Z \parallel \text{trunc}_{r'}(C)$ 
20:   if  $|Z| < \nu$  then
21:      $(B, C) \leftarrow F_2(B, C)$ 
22:      $Z \leftarrow Z \parallel \text{trunc}_{r'}(C)$ 
23:   end if
24: end while
25: return  $Z[0 : \nu - 1]$ 

```

2.3 Message Padding

To parse the input into r -bit message blocks unambiguously, Duet applies 10^* padding to the input message $M \in \{0, 1\}^*$. The padded message is obtained as follows.

First, append a single bit 1 to M . Then append k zero bits, where k is the smallest non-negative integer satisfying

$$|M| + 1 + k \equiv 0 \pmod{r}.$$

The resulting padded message is denoted by

$$M^* = M \parallel 1 \parallel 0^k.$$

It is then parsed into r -bit blocks as

$$M^* = M_0 \parallel M_1 \parallel \cdots \parallel M_\ell, \quad M_i \in \{0, 1\}^r.$$

This padding rule is applied even when $|M|$ is already a multiple of r . In that case, an additional full block $1 \parallel 0^{r-1}$ is appended. This guarantees that the padding is injective and prevents ambiguity caused by trailing zero bits.

2.4 Absorbing Phase

The absorbing phase processes the padded blocks $M_0, \dots, M_\ell$ sequentially. Each block triggers two serial permutation calls, first to f_1 and then to f_2 , and both calls apply the capacity feed-forward rule defined above.

State initialization

Before processing the first message block, the internal state is initialized as

$$A \leftarrow 0^r, \quad B \leftarrow 0^r, \quad C \leftarrow 0^c$$

Iterative absorption

For each padded block M_i , where $0 \leq i \leq \ell$, let (A_0, B_0, C_0) be the current state before processing the block. The first-stage call absorbs M_i into the first rate branch:

$$(A_1, C_1) = F_1(A_0 \oplus M_i, C_0).$$

The second-stage call absorbs the transformed block $\sigma(M_i)$ into the second rate branch, using the updated capacity value C_1 :

$$(B_1, C_2) = F_2(B_0 \oplus \sigma(M_i), C_1).$$

The state is then updated as

$$(A, B, C) \leftarrow (A_1, B_1, C_2).$$

After the final block M_ℓ has been processed, the resulting state (A, B, C) becomes the initial state of the squeezing phase.

2.5 Squeezing Phase and Output Mechanism

After the absorbing phase, Duet enters the squeezing phase with the finalized state (A, B, C) . For a selected Duet- h instance, the squeezing rate is derived from the digest length and fixed as $r' = h \leq c$. Thus, each complete squeezing step extracts an h -bit output fragment from the capacity branch. Thus, every complete squeezing step produces an h -bit output fragment from the capacity branch.

Let $\nu \geq 1$ be the requested total output length, and define

$$t = \left\lceil \frac{\nu}{h} \right\rceil$$

as the number of output fragments required. The first fragment is extracted directly from the finalized capacity state:

$$z_0 = \text{trunc}_h(C).$$

In particular, the fixed-length h -bit digest is defined as

$$H_h(M) = \text{Duet}_h(M, h) = z_0 = \text{trunc}_h(C).$$

If $t > 1$, additional fragments are generated through empty alternating updates. No message block is injected during these updates. For $j = 1, \dots, t - 1$, the state is updated as follows:

$$\begin{cases} (A, C) \leftarrow F_1(A, C), & \text{if } j \text{ is odd,} \\ (B, C) \leftarrow F_2(B, C), & \text{if } j \text{ is even.} \end{cases}$$

The rate branch not involved in the current update remains unchanged. After the corresponding update, the next output fragment is

$$z_j = \text{trunc}_h(C).$$

The generated fragments are concatenated as

$$Z^* = z_0 \parallel z_1 \parallel \dots \parallel z_{t-1}.$$

The requested output is the first ν bits of Z^* :

$$\text{Duet}_h(M, \nu) = Z^*[0 : \nu - 1].$$

When $1 \leq \nu < h$, the output is the first ν bits of the fixed-length digest. When $\nu = h$, this definition gives the fixed-length Duet- h digest. When $\nu > h$, it provides an extendable-output interface by continuing the alternating squeezing process. A longer requested output does not increase the claimed security strength beyond that of the selected Duet- h instance.

2.6 Specification of the Permutation f

This subsection specifies the permutations f_1 and f_2 employed by Duet. We write $\alpha \in \{1, 2\}$ for the permutation index, so that f_α denotes either f_1 or f_2 . We write $\beta \in \{960, 1920\}$ for the permutation width, and use $f\text{-}\beta$ to denote the corresponding width variant. For the 512-bit digest variant, $f\text{-}960$ operates on a 960-bit state, represented as a 5×3 array of 64-bit words. For the 768-bit and 1024-bit digest variants, $f\text{-}1920$ operates on a 1920-bit state, organized as a 5×6 array of 64-bit words. For both widths, the state is denoted by

$$X = (X_{i,j})_{0 \leq i < 5, 0 \leq j < m},$$

where $m = 3$ for $f\text{-}960$ and $m = 6$ for $f\text{-}1920$. Each $X_{i,j} \in \{0, 1\}^{64}$, so the total state size is $320m$ bits.

For a fixed $k \in \{0, \dots, 63\}$, the bits at bit position k form a $5m$ -bit slice

$$X^{(k)} = \begin{pmatrix} x_{0,0,k} & x_{0,1,k} & \cdots & x_{0,m-1,k} \\ x_{1,0,k} & x_{1,1,k} & \cdots & x_{1,m-1,k} \\ x_{2,0,k} & x_{2,1,k} & \cdots & x_{2,m-1,k} \\ x_{3,0,k} & x_{3,1,k} & \cdots & x_{3,m-1,k} \\ x_{4,0,k} & x_{4,1,k} & \cdots & x_{4,m-1,k} \end{pmatrix},$$

where $x_{i,j,k}$ denotes the bit of $X_{i,j}$ at bit position k . The indices (i, j, k) specify the row, column, and bit position, respectively.

The $f\text{-}\beta$ permutation consists of N_β rounds, where $N_{960} = 12$ and $N_{1920} = 18$. For $\gamma = 0, \dots, N_\beta - 1$, the round function ϕ_γ consists of a Substitute-Column layer SC, a Mix-Slice layer MS, an Add-Constant layer AC, and a Mix-Lane layer ML. Thus,

$$\phi_\gamma = \text{ML} \circ \text{AC} \circ \text{MS} \circ \text{SC}.$$

Substitute-Column layer SC. For each column j and bit position k , the five bits

$$x_{0,j,k}, x_{1,j,k}, x_{2,j,k}, x_{3,j,k}, x_{4,j,k}$$

are used as the input to a 5-bit S-box. More precisely, these bits are viewed as a 5-bit word

$$u = (x_{0,j,k} \| x_{1,j,k} \| x_{2,j,k} \| x_{3,j,k} \| x_{4,j,k}),$$

where $x_{0,j,k}$ is the most significant bit. The S-box is given in Table 1.

Table 1: 5-bit S-box used in the SC layer

u		00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
$S(u)$		1A	09	10	17	07	08	15	0E	19	1F	1E	0C	1B	01	04	0A
u		10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
$S(u)$		05	13	0D	0F	16	1C	06	18	11	12	14	03	1D	02	00	0B

The S-box can also be implemented by the following Boolean circuit, where $\wedge$, $\oplus$, and $\neg$ denote bitwise AND, XOR, and NOT, respectively:

Table 2: Boolean circuit implementation of the 5-bit S-box

$t_0 = x_3 \wedge x_4$	$t_{13} = t_{11} \oplus t_{12}$	$t_{26} = x_2 \oplus x_0$	$t_{39} = t_{16} \oplus t_5$
$t_1 = x_2 \wedge x_4$	$t_{14} = \neg t_{13} (y_0)$	$t_{27} = t_0 \oplus t_1$	$t_{40} = t_{36} \oplus t_{37}$
$t_2 = x_2 \wedge x_3$	$t_{15} = x_1 \wedge x_3$	$t_{28} = t_3 \oplus t_{15}$	$t_{41} = t_{38} \oplus t_{39}$
$t_3 = x_1 \wedge x_4$	$t_{16} = x_0 \wedge x_2$	$t_{29} = t_4 \oplus t_{25}$	$t_{42} = t_{40} \oplus t_{41}$
$t_4 = x_1 \wedge x_2$	$t_{17} = x_3 \oplus x_2$	$t_{30} = t_{16} \oplus t_5$	$t_{43} = \neg t_{42} (y_3)$
$t_5 = x_0 \wedge x_1$	$t_{18} = x_0 \oplus t_1$	$t_{31} = t_{26} \oplus t_{27}$	$t_{44} = x_4 \oplus x_2$
$t_6 = x_4 \oplus x_2$	$t_{19} = t_2 \oplus t_{15}$	$t_{32} = t_{28} \oplus t_{29}$	$t_{45} = x_1 \oplus x_0$
$t_7 = x_0 \oplus t_0$	$t_{20} = t_4 \oplus t_{16}$	$t_{33} = t_{30} \oplus t_{31}$	$t_{46} = t_3 \oplus t_{15}$
$t_8 = t_1 \oplus t_2$	$t_{21} = t_{17} \oplus t_{18}$	$t_{34} = t_{32} \oplus t_{33} (y_2)$	$t_{47} = t_4 \oplus t_{25}$
$t_9 = t_3 \oplus t_4$	$t_{22} = t_{19} \oplus t_{20}$	$t_{35} = x_0 \wedge x_3$	$t_{48} = t_5 \oplus t_{44}$
$t_{10} = t_5 \oplus t_6$	$t_{23} = t_{21} \oplus t_{22}$	$t_{36} = x_4 \oplus x_3$	$t_{49} = t_{45} \oplus t_{46}$
$t_{11} = t_7 \oplus t_8$	$t_{24} = \neg t_{23} (y_1)$	$t_{37} = x_1 \oplus x_0$	$t_{50} = t_{47} \oplus t_{48}$
$t_{12} = t_9 \oplus t_{10}$	$t_{25} = x_0 \wedge x_4$	$t_{38} = t_4 \oplus t_{35}$	$t_{51} = t_{49} \oplus t_{50} (y_4)$

Mix-Slice layer MS. The MS layer is a linear transformation on the $5m$ 64-bit words of the state. The words are ordered column by column as

$$\mathbf{X} = (X_{0,0}, X_{1,0}, \dots, X_{4,0}, X_{0,1}, X_{1,1}, \dots, X_{4,1}, \dots, X_{0,m-1}, X_{1,m-1}, \dots, X_{4,m-1})^\top.$$

The output is $\mathbf{Y} = M_{5m} \cdot \mathbf{X}$, and the state is updated by setting $X_{i,j} \leftarrow Y_{5j+i}$. In the matrices below, entry 1 means the 64×64 identity block and entry 0 means the 64×64 zero block.

For f -960, M_{15} is the following 15×15 block matrix:

$$M_{15} = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

For f -1920, M_{30} is the 30×30 right-circulant block matrix whose first row is

$$v = (1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0).$$

Each subsequent row is obtained by a cyclic right shift by one position.

Add-Constant layer AC. The round constants are denoted by $\mathcal{C}_{\alpha,\beta,\gamma}$. In round γ , the corresponding constant is injected into the first word:

$$X_{0,0} \leftarrow X_{0,0} \oplus \mathcal{C}_{\alpha,\beta,\gamma}.$$

The constants for f -960 are derived from the fractional part of π . The two columns below give $\mathcal{C}_{1,960,\gamma}$ and $\mathcal{C}_{2,960,\gamma}$, respectively:

Table 3: Round constants for f -960

γ	$\mathcal{C}_{1,960,\gamma}$	$\mathcal{C}_{2,960,\gamma}$
0	0x243f6a8885a308d3	0xba7c9045f12c7f99
1	0x13198a2e03707344	0x24a19947b3916cf7
2	0xa4093822299f31d0	0x0801f2e2858efc16
3	0x082efa98ec4e6c89	0x636920d871574e69
4	0x452821e638d01377	0xa458fea3f4933d7e
5	0xbe5466cf34e90c6c	0x0d95748f728eb658
6	0xc0ac29b7c97c50dd	0x718bcd5882154aee
7	0x3f84d5b5b5470917	0x7b54a41dc25a59b5
8	0x9216d5d98979fb1b	0x9c30d5392af26013
9	0xd1310ba698dfb5ac	0xc5d1b023286085f0
10	0x2ffd72dbd01adfb7	0xca417918b8db38ef
11	0xb8e1afed6a267e96	0x8e79dcb0603a180e

The constants for f -1920 are derived from the fractional part of e . The two columns below give $\mathcal{C}_{1,1920,\gamma}$ and $\mathcal{C}_{2,1920,\gamma}$, respectively:

Table 4: Round constants for f -1920

γ	$\mathcal{C}_{1,1920,\gamma}$	$\mathcal{C}_{2,1920,\gamma}$
0	0xb7e151628aed2a6a	0x753d0a8f78e537d2
1	0xbf7158809cf4f3c7	0xb95bb79d8dcaec64
2	0x62e7160f38b4da56	0x2c1e9f23b829b5c2
3	0xa784d9045190cfef	0x780bf38737df8bb3
4	0x324e7738926cfbe5	0x00d01334a0d0bd86
5	0xf4bf8d8d8c31d763	0x45cbfa73a6160ffe
6	0xda06c80abb1185eb	0x393c48cbbbca060f
7	0x4f7c7b5757f59584	0x0ff8ec6d31beb5cc
8	0x90cfd47d7c19bb42	0xeed7f2f0bb088017
9	0x158d9554f7b46bce	0x163bc60df45a0ecb
10	0xd55c4d79fd5f24d6	0x1bcd289b06cbbfea
11	0x613c31c3839a2ddf	0x21ad08e1847f3f73
12	0x8a9a276bcfbfa1c8	0x78d56ced94640d6e
13	0x77c56284dab79cd4	0xf0d3d37be67008e1
14	0xc2b3293d20e9e5ea	0x86d1bf275b9b241d
15	0xf02ac60acc93ed87	0xeb64749a47dfdfb9
16	0x4422a52ecb238fee	0x6632c3eb061b6472
17	0xe5ab6add835fd1a0	0xbbf84c26144e49c2

Mix-Lane layer ML. Each word is updated independently. With the linear index

$$t = 5j + i,$$

the update is

$$X_{i,j} \leftarrow (X_{i,j} \lll t) \oplus (X_{i,j} \lll (t + 21)) \oplus (X_{i,j} \lll (t + 43)).$$

For f -960, the index range is $0 \leq t < 15$, and for f -1920, it is $0 \leq t < 30$.

The complete procedure for the f - β permutation is given in Algorithm 2.

Algorithm 2 Permutation f - β

Require: State X , permutation index $\alpha \in \{1, 2\}$, permutation width $\beta \in \{960, 1920\}$

Ensure: Updated state X

- 1: **for** $\gamma = 0$ to $N_\beta - 1$ **do**
 - 2: $X \leftarrow \text{SC}(X)$
 - 3: $X \leftarrow \text{MS}(X)$
 - 4: $X_{0,0} \leftarrow X_{0,0} \oplus \mathcal{C}_{\alpha,\beta,\gamma}$
 - 5: $X \leftarrow \text{ML}(X)$
 - 6: **end for**
 - 7: **return** X
-

3 Design Rationale

The design goal of Duet is to provide a permutation-based hash construction, flexible output length, and strong security for selected digest sizes. Although Duet offers an extendable-output interface, its main security claims are stated for the first h output bits, which serve as the h -bit digest. The construction is designed around three principles: shared-capacity state organization, serial alternating permutation calls, and capacity feed-forward. Together, these choices define the state evolution, keep each underlying permutation call at width $b = r + c$, and support the required collision, preimage, and second-preimage security levels.

3.1 Shared-Capacity Alternation and Capacity Feed-Forward

Duet organizes two r -bit rate branches around a shared c -bit capacity chain. During the absorption of each message block, f_1 first updates the upper branch and the capacity, after which the resulting capacity value is used by f_2 to update the middle branch. The two rate branches are therefore serially coupled rather than independent. The logical state has $2r + c$ bits, while each permutation operates on $r + c$ bits.

The shared-capacity mechanism is further combined with capacity feed-forward. For a permutation call satisfying

$$f_s(R \parallel C) = R' \parallel D, \quad s \in \{1, 2\},$$

the next capacity value is defined as

$$C' = D \oplus C.$$

Instead of using D , the capacity part of the permutation output, directly as the next capacity value, the feed-forward rule makes the capacity evolution depend jointly on the previous capacity C and the permutation output D . Consequently, obtaining a prescribed next capacity value requires satisfying a relation involving both the input capacity and the permutation output. This update has a truncated Davies-Meyer-like form and provides the structural basis for the preimage and second-preimage analysis of the capacity-derived digest [6, 7].

During squeezing, Duet follows the same design principle. The first output fragment is extracted from the finalized capacity value, and further fragments are generated through alternating empty updates using f_1 and f_2 . This preserves the structural consistency between absorption and squeezing, makes the public digest originate from the capacity component governing the main security parameters, and retains an XOF interface for variable-length output.

3.2 Three-dimensional Permutation f

Permutations f_1 and f_2 are core components used in Duet hash function. When designing these permutations, we consistently take into account the speed of confusion and diffusion, implementation performance (e.g. latency), and hardware cost (e.g. area). The specific design principles are as follows.

Design principle of overall structure: Permutation $f \in \{f_1, f_2\}$ employs a three-dimensional diffusion architecture. By alternating between non-linear and linear components, this design aims to achieve rapid confusion and diffusion while maintaining low gate depth in the round function implementation. All operations are processed in 64-bit units, facilitating efficient implementation on mainstream 64-bit software platforms. Round constants are integrated into each round function to break the algorithmic symmetry and thwart slide attacks. Permutations f_1 and f_2 adopt similar round function structures, differing solely in the specific round constants employed.

Design principle of the nonlinear S-boxes. The non-linear layer is composed of parallel 5-bit S-boxes. We search an S-box that satisfies specific criteria: a differential uniformity of 2, an algebraic degree of 2, the absence of fixed points and exclusion of differential patterns $\alpha \rightarrow \alpha$, where the hamming weight $hw(\alpha) = 1$. Through implementation optimization, the gate depth of this S-box can be reduced to 5. It is also highly suitable for bit-sliced implementation. Consequently, it achieves excellent cryptographic properties alongside favorable implementation performance.

Design principle of the linear Mix-Slice layer. The MS layer is constructed using bit-level circulant matrices, implemented via cyclic shifts and XOR operations to minimize the critical path length. For the f -960 variant, the linear layer MS is organized as a 3×3 block matrix operating on 5-bit blocks, designed to process the 15-bit state within a single slice. To ensure sufficient cryptographic security, the block-wise matrix is designed to achieve a branch number of 4, while its corresponding 15×15 bit-wise matrix achieves a branch number of 8,

with an implementation gate depth of 3. Similarly, for the f -1920 variant, the linear layer MS adopts a 6×6 block matrix structure with 5-bit blocks to process the 30-bit state per slice. To guarantee robust cryptographic properties, the block-wise matrix targets a branch number of 6, and the corresponding 30×30 bit-wise matrix achieves a branch number of 12, with a gate depth of 4.

Design principle of the linear Mix-Lane layer. In the ML layer, each lane is updated using a bit-level circulant matrix, which is implemented via cyclic shifts and XOR operations. To strike a balance between cryptographic security and implementation efficiency, the branch number is set to 4. Furthermore, to enhance diffusion, all circulant matrices are designed to be distinct. However, to maintain favorable implementation performance, the circulant matrices across different lanes are derived from one another through simple cyclic shift operations.

3.3 Parameter Selection and Security Targets

For an h -bit digest, Duet targets $h/2$ -bit collision security, h -bit preimage security, and h -bit second-preimage security. Under the assumptions and bounds stated in Section 4.2, sufficient parameter conditions are

$$c \geq h$$

for h -bit preimage security, and

$$c \geq h + \log_2(2N_{\max})$$

for h -bit second-preimage security, where $N_{\max}$ is the maximum number of padded message blocks.

Let $L_{\max}$ denote the maximum supported message length in bits. Under the 10* padding rule,

$$N_{\max} = \left\lceil \frac{L_{\max} + 1}{r} \right\rceil.$$

The evaluation criteria require support for messages of length at least $2^{64} - 1$ bits. In the concrete parameter verification, we set $L_{\max}$ to this value, which covers the required message-length range.

The parameter selection follows a digest-aware state allocation principle. The capacity c is chosen to meet the security requirements for the target digest length, including the required long-message margin for second-preimage resistance. The remaining state bits are then assigned to the rate r , since a larger rate improves the message absorption throughput. In this way, the state size and the rate-capacity split are adapted to the target digest length, rather than forcing all digest lengths to use the same state allocation.

More concretely, we define three instances: Duet-512, Duet-768, and Duet-1024. Duet-512 and Duet-1024 serve as the two main instances, maintaining the same rate-capacity ratio $r : c = 2 : 3$ with permutation widths of 960 bits and 1920 bits, respectively. Duet-768 is defined as an intermediate security-level instance. It uses the same 1920-bit permutation as Duet-1024, but adopts a different rate-capacity partition. The selected parameter sets are summarized in Table 5.

Table 5: Selected parameter sets for the supported digest lengths

Instance	Digest length h	Rate r	Capacity c	Permutation width $b = r + c$
Duet-512	512	384	576	960
Duet-768	768	1088	832	1920
Duet-1024	1024	768	1152	1920

For the maximum supported message length of $2^{64} - 1$ bits, the largest value of $\log_2(2N_{\max})$ among the selected parameter sets is less than 57. In contrast, the smallest capacity margin $c - h$

is 64 bits. Therefore, all supported digest lengths satisfy $c \geq h$ and $c \geq h + \log_2(2N_{\max})$, and hence meet the preimage and second-preimage parameter conditions under the security model in Section 4.2.

4 Security Claims and Analysis

Although Duet supports capacity-based squeezing and variable-length outputs, we focus on the security analysis of fixed h -bit digest obtained from the first h squeezing bits. Following the evaluation criteria, we separate theoretical security, covering models, assumptions, and bounds, from concrete security, which instantiates these bounds for selected digest lengths.

4.1 Security Claims

For each supported digest length $h \in \{512, 768, 1024\}$, Duet is designed to match the standard generic security targets for an h -bit hash output. In the classical setting, the targets are $\frac{h}{2}$ -bit collision security, h -bit preimage security, and h -bit second-preimage security for the supported message-length range. In the quantum setting, the intended benchmarks are the best known generic quantum attack costs for a random h -bit hash output, namely $\frac{h}{3}$ -bit collision security and $\frac{h}{2}$ -bit preimage and second-preimage security. The notions of collision resistance, preimage resistance, and second-preimage resistance follow the standard hash-function security definitions by Rogaway and Shrimpton [13].

The theoretical claims are made in the ideal-permutation model, assuming that f_1 and f_2 are modeled as two independent ideal permutations. Section 4.2 gives the generic advantage bounds and the corresponding parameter conditions. Section 4.3 then instantiates these bounds for Duet-512, Duet-768, and Duet-1024, and summarizes the resulting concrete security levels.

4.2 Theoretical Security

4.2.1 Model and Assumptions

For $h \leq c$, let $C_M \in \{0, 1\}^c$ denote the finalized capacity value obtained after absorbing the padded representation of a message M . The corresponding h -bit digest is defined as

$$H_h(M) = \text{trunc}_h(C_M).$$

The 10^* padding rule is injective; hence distinct messages induce distinct padded block sequences.

The security bounds below are established in the ideal-permutation model. Specifically, f_1 and f_2 are modeled as independent uniformly random permutations on $\{0, 1\}^b$, where $b = r + c$. If both call domains are instantiated using a single concrete permutation, they must be separated by an appropriate domain-separation mechanism. The resulting security claims nevertheless remain conditional on the ideal-permutation assumption.

Under the capacity feed-forward rule defined in Section 2.1, a fresh permutation call produces a capacity value whose h -bit truncation is unpredictable to the adversary.

The proofs distinguish two generic types of successful events. First, in the absence of an internal reconnection, success occurs directly in the h -bit digest space: this is a target-matching event for preimage-type attacks and a birthday-matching event for collision attacks. Second, an adversary may attempt an internal reconnection, in which one computation path merges into a state reached by another path and can thereafter follow the same suffix computation. Any such reconnection necessarily entails a match of a c -bit capacity value. Counting all such capacity matches therefore gives a conservative upper bound on internal reconnection events. The following analysis bounds the digest-level and reconnection events separately for each security notion.

4.2.2 Indifferentiability Background of the Alternating Structure

The indifferentiability framework was introduced by Maurer, Renner, and Holenstein [15]. It provides a methodology for comparing an idealized construction with a random oracle while allowing the adversary to access the underlying ideal primitive. Within this framework, the standard sponge construction was proven indifferentiable from a random oracle up to the birthday bound in its capacity parameter [14].

Before discussing the collision, preimage, and second-preimage bounds of Duet, we first clarify the structural security background of its alternating two-branch schedule. The overall construction of Duet is not identical to the Alternating Sponge proposed in the previous work [12]: Duet further introduces capacity feed-forward updates and derives its output from the capacity branch. Therefore, the indifferentiability analysis of Alternating Sponge should be understood as an important structural security background and design rationale for Duet.

The core scheduling idea of Duet follows the Alternating Sponge framework: the two rate branches are alternately paired with the same shared capacity branch as inputs to the underlying permutations, and the shared capacity is updated after each permutation call. The previous work proves that this alternating two-branch schedule achieves beyond-birthday-bound indifferentiability security in the ideal-permutation model, with a generic security threshold of $2^{\min\{\frac{2c}{3}, \frac{r+c}{2}\}}$, i.e., $\min\{\frac{2c}{3}, \frac{r+c}{2}\}$ bits of security. This result indicates that the alternating schedule provides a strong generic-security foundation for sponge-like constructions, and supports its use as the structural basis of Duet.

Based on this structural motivation, we next analyze the concrete collision, preimage, and second-preimage bounds of Duet under the classical attack model, taking into account its capacity feed-forward and capacity-based output mechanisms.

4.2.3 Classical Advantage Bounds

Let Q denote the total number of forward and inverse permutation queries made by the adversary to f_1 and f_2 . All permutation calls required to evaluate candidate messages are included in Q . In particular, evaluating a padded message of N blocks requires $2N$ permutation queries during absorption. Let $N_{\max}$ denote the maximum number of padded r -bit message blocks allowed by the supported message-length range. Under the model described in 4.2.1, the classical advantage bounds are summarized in Table 6.

Table 6: Theoretical classical advantage bounds for an h -bit digest

Property	Classical advantage bound	Condition controlling the bound
Collision resistance	$O(\frac{Q^2}{2^h}) + O(\frac{Q^2}{2^c})$	$h \leq c$
Preimage resistance	$O(\frac{Q}{2^h})$	$h \leq c$
Second-preimage resistance	$O(\frac{Q}{2^h}) + O(\frac{2QN_{\max}}{2^c})$	$c \geq h + \log_2(2N_{\max})$

Collision resistance. For a message M , let $X = R \parallel C$ be the input to the final f_2 call in the absorbing phase, and write $f_2(X) = R' \parallel D$. By the capacity feed-forward rule,

$$H_h(M) = \text{trunc}_h(C \oplus D) = \text{trunc}_h(\text{right}_c(X \oplus f_2(X))).$$

Thus each digest can be viewed as the h -bit truncation of a Davies-Meyer-type capacity feed-forward value [6].

A collision between two distinct messages can occur in two ways. First, if the final f_2 inputs are distinct, the two truncated feed-forward values behave like two random h -bit strings, so the birthday term is $O(\frac{Q^2}{2^h})$. Second, two different absorbing paths may reconnect before the final

call. Such an internal reconnection requires matching a c -bit capacity value, and over $O(Q^2)$ relevant pairs this contributes $O(\frac{Q^2}{2^c})$. Therefore

$$\text{Adv}_{\text{Duet}}^{\text{coll}}(Q) \leq O(\frac{Q^2}{2^h}) + O(\frac{Q^2}{2^c}).$$

When $h \leq c$, the internal-reconnection term is no larger than the final-output birthday term.

Preimage resistance. For a target digest $Z \in \{0,1\}^h$, a successful preimage attack must produce a message M whose final capacity feed-forward value satisfies

$$\text{trunc}_h(C \oplus D) = Z.$$

In the ideal-permutation model, for a fresh final f_2 input $X = R \parallel C$, the output capacity part D is unpredictable and uniformly distributed over the remaining output space. Since C is fixed before the query, $C \oplus D$ is also random from the adversary's viewpoint. Hence each fresh forward or inverse permutation query produces the target digest with probability about 2^{-h} . The same reasoning applies to fresh inverse-permutation queries. By the union bound,

$$\text{Adv}_{\text{Duet}}^{\text{pre}}(Q) \leq O(\frac{Q}{2^h}).$$

The condition $h \leq c$ is necessary because the digest is extracted from a capacity value; if $h > c$, a single capacity feed-forward output cannot provide an h -bit target space.

Second-preimage resistance. Long-message effects are important in second-preimage analysis, as demonstrated by generic attacks on iterated hash functions [8].

In the second-preimage setting, the adversary is given a challenge message M and may compute its entire internal path. Suppose the padded challenge message contains N blocks. Since each block triggers two permutation calls, one to f_1 and one to f_2 , the challenge path contains at most $2N$ conservative reconnection positions.

There are two generic routes. The first route is to hit the final digest directly, which contributes $O(\frac{Q}{2^h})$ by the preimage argument. The second route is to reconnect to one of the $2N$ internal positions on the challenge path. Reconnection to a specified position requires matching the corresponding c -bit capacity value, so the total reconnection contribution is $O(\frac{2QN}{2^c})$. Therefore

$$\text{Adv}_{\text{Duet}}^{2\text{pre}}(Q, N) \leq O(\frac{Q}{2^h}) + O(\frac{2QN}{2^c}).$$

For all supported messages, we have $N \leq N_{\max}$, and hence the reconnection term is upper-bounded by $O(\frac{2QN_{\max}}{2^c})$. To keep this long-message reconnection term no larger than the direct h -bit digest-hitting term $O(\frac{Q}{2^h})$, it is sufficient to require

$$\frac{2N_{\max}}{2^c} \leq \frac{1}{2^h},$$

or equivalently,

$$c \geq h + \log_2(2N_{\max}).$$

This length-dependent capacity condition is analogous to the $c - \log_2(\alpha)$ second-preimage bounds obtained for feed-forward sponge constructions [6].

4.2.4 Quantum Generic Bounds

We consider a quantum attack model in which the adversary may run quantum algorithms and make quantum queries to the idealized public permutations. The construction itself remains classical; the quantum setting only strengthens the adversary. The estimates below are generic black-box estimates rather than full quantum security reductions. They assume that the h -bit digest behaves as a random h -bit value and that the instantiated permutations do not expose exploitable algebraic structure, hidden periods, or other special structure enabling a dedicated quantum attack.

For preimage and second-preimage search, the relevant generic problem is unstructured search over a space of effective size 2^h . Grover’s algorithm reduces the query complexity from 2^h to

$$\sqrt{2^h} = 2^{h/2}$$

queries [9]. For collision search, the Brassard–Høyer–Tapp algorithm finds collisions in a random function with query complexity on the order of the cube root of the output-space size [10]. For an h -bit digest, this gives

$$(2^h)^{1/3} = 2^{h/3}.$$

Known quantum lower bounds for the collision problem show that this cube-root behavior is the correct generic black-box scale [11]. Therefore, the quantum benchmarks used for concrete evaluation are $2^{h/3}$ for collision resistance and $2^{h/2}$ for preimage and second-preimage resistance.

4.2.5 Additional Structural Security Considerations

Resistance to Length-Extension Attacks. A typical Merkle–Damgård length-extension attack relies on a crucial condition: the public digest is exactly the final chaining value, so an attacker can use the digest as a new initial state and continue the computation on an extended message. This condition does not hold for Duet.

Let the internal state after absorbing a message M be (A_M, B_M, C_M) . To absorb additional message blocks, the algorithm requires the complete state A_M , B_M , and C_M in order to perform the subsequent two-stage updates. In contrast, the fixed-length digest only reveals

$$H_h(M) = \text{trunc}_h(C_M).$$

The digest contains neither the two rate branches A_M, B_M , nor, in general, the full capacity branch C_M . Even when $h = c$, the two rate branches remain hidden.

Therefore, an attacker cannot continue the computation from $H_h(M)$ alone. Consequently, in the ideal-permutation model, standard Merkle–Damgård-style length-extension attacks are not directly applicable to the fixed-length digest instances of Duet. Any related attack would have to go beyond direct continuation from the digest, for example by recovering hidden internal state information, constructing an internal reconnection, or finding a second preimage.

4.3 Concrete Security

The concrete-security layer instantiates the theoretical bounds for selected digest lengths $h \in \{512, 768, 1024\}$. According to the evaluation criteria, a hash algorithm with digest length h should provide at least $\frac{h}{2}$ bits of classical collision security, h bits of classical preimage security, and h bits of classical second-preimage security. In the quantum setting, the reference benchmarks are the best known generic quantum attack complexities for an h -bit hash output.

For second-preimage security, the concrete capacity must account for the maximum supported message length. Let $N_{\max}$ be the maximum number of padded message blocks under the

10* padding rule. Since the submission supports messages of at most $L_{\max} = 2^{64} - 1$ bits, we have

$$N_{\max} = \left\lceil \frac{L_{\max} + 1}{r} \right\rceil = \left\lceil \frac{2^{64}}{r} \right\rceil.$$

Thus the concrete second-preimage condition is $c \geq h + \log_2(2N_{\max})$. For the selected parameter sets, the rate satisfies $r \geq 384$. Hence, under the maximum supported message length $L_{\max} = 2^{64} - 1$, the length-dependent term $\log_2(2N_{\max})$ is uniformly upper-bounded by 57. Since the smallest capacity margin among Duet-512, Duet-768, and Duet-1024 is $c - h = 64$, all supported digest lengths satisfy $c \geq h + \log_2(2N_{\max})$, as well as the basic condition $c \geq h$.

Table 7: Concrete classical and quantum generic security strengths

Instance	Classical			Quantum		
	Collision	Preimage	2nd-preimage	Collision	Preimage	2nd-preimage
Duet-512	256	512	512	170.7	256	256
Duet-768	384	768	768	256	384	384
Duet-1024	512	1024	1024	341.3	512	512

Therefore, the selected parameters for Duet-512, Duet-768, and Duet-1024 meet the stated classical security requirements for collision, preimage, and second-preimage resistance. Their quantum security levels are evaluated against the standard generic quantum attack costs for random h -bit hash outputs: $2^{\frac{h}{3}}$ for collision search and $2^{\frac{h}{2}}$ for preimage and second-preimage search.

4.4 Security of Permutation f

4.4.1 Security of the S-box

The f -960 and f -1920 permutations adopt the identical 5-bit S-box introduced in Table 1 for their non-linear SC layers. The difference distribution table (DDT) and linear approximation table (LAT) characterizing its differential and linear behaviors are listed in Table 8 and Table 9, respectively. Statistical analysis on the tables reveals that the S-box achieves a maximum differential probability of $2/32 = 2^{-4}$ and a maximum absolute linear correlation magnitude of $4/16 = 2^{-2}$. In addition, the differential patterns: $01 \rightarrow 01$, $02 \rightarrow 02$, $04 \rightarrow 04$, $08 \rightarrow 08$ and $10 \rightarrow 10$ are avoided to resist underlying differential distinguishing attacks.

Table 8: Difference Distribution Table (DDT) of the 5-bit S-box (32×32)

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
00	32	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
01	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2
02	0	0	0	0	2	2	2	2	2	2	2	2	0	0	0	0	2	2	2	2	0	0	0	0	0	0	0	0	2	2	2	2
03	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0
04	0	2	2	0	0	2	2	0	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	0	2	2	0	0	2	2	0
05	0	0	2	2	2	2	0	0	2	2	0	0	0	0	2	2	0	0	2	2	2	2	0	0	2	2	0	0	0	0	2	2
06	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2
07	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
08	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0
09	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2
0A	0	0	2	2	2	2	0	0	2	2	0	0	0	0	2	2	2	2	0	0	0	0	2	2	0	0	2	2	2	2	0	0
0B	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2
0C	0	2	2	0	2	0	0	2	2	0	0	2	0	2	2	0	0	2	2	0	2	0	0	2	2	0	0	2	0	2	2	0
0D	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2	2	2	2	2	0	0	0	0	2	2	2	2	0	0	0	0
0E	0	0	2	2	2	2	0	0	0	0	2	2	2	2	0	0	2	2	0	0	0	0	2	2	2	2	0	0	0	0	2	2
0F	0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	0	0	0	0	0	0	0	0
10	0	2	0	2	2	0	2	0	2	0	2	0	0	2	0	2	0	2	0	2	2	0	2	0	2	0	2	0	0	2	0	2
11	0	0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	2
12	0	2	0	2	2	0	2	0	2	0	2	0	0	2	0	2	2	0	2	0	0	2	0	2	0	2	0	2	0	2	0	2
13	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0	2	0	2	0	0	2	0	2	2	0	2	0	0	2	0	2
14	0	2	2	0	2	0	0	2	0	2	2	0	2	0	0	2	2	0	0	2	0	2	2	0	2	0	0	2	0	2	2	0
15	0	0	2	2	0	0	2	2	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0	0	0	2	2	0	0	2	2
16	0	0	2	2	0	0	2	2	0	0	2	2	0	0	2	2	2	2	0	0	2	2	0	0	2	2	0	0	2	2	0	0
17	0	2	2	0	0	2	2	0	2	0	0	2	2	0	0	2	0	2	2	2	0	2	2	0	2	0	0	2	2	0	0	2
18	0	0	2	2	2	2	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2	0	0
19	0	2	0	2	0	2	0	2	2	0	2	0	2	0	2	0	0	2	0	2	0	2	0	2	2	0	2	0	2	0	2	0
1A	0	2	0	2	0	2	0	2	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	0	2	0	2	0	2	0	2
1B	0	0	2	2	0	0	2	2	2	2	0	0	2	2	0	0	0	0	2	2	0	0	2	2	2	2	0	0	2	2	0	0
1C	0	2	0	2	0	2	0	2	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0
1D	0	0	0	0	2	2	2	2	2	2	2	2	0	0	0	0	0	0	0	0	0	2	2	2	2	2	2	2	0	0	0	0
1E	0	2	2	0	2	0	0	2	2	0	0	2	0	2	2	0	2	0	0	2	0	2	2	0	0	2	2	0	2	0	0	2
1F	0	2	2	0	2	0	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0	0	2

Table 9: Linear Approximation Table (LAT) of the 5-bit S-box (32×32)

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	
00	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
01	0	0	4	-4	-4	0	0	4	4	4	4	4	-4	0	4	0	-4	0	0	4	-4	4	0	0	4	0	-4	0	0	0	0	0	
02	0	-4	0	-4	4	0	0	-4	0	0	-4	-4	0	0	0	0	-4	0	4	0	0	4	4	0	4	-4	0	0	-4	4	-4	-4	
03	0	-4	-4	0	0	0	0	0	-4	-4	0	0	-4	0	4	0	0	0	-4	4	-4	0	-4	0	0	-4	4	0	4	4	-4	4	
04	0	-4	0	0	0	0	0	-4	0	0	0	4	4	0	-4	4	-4	-4	4	0	-4	0	-4	4	-4	0	-4	-4	0	0	0	4	
05	0	4	4	-4	4	0	0	0	-4	-4	4	0	0	0	-4	0	-4	0	-4	-4	-4	0	4	-4	4	0	0	0	-4	0	0	-4	
06	0	0	0	-4	4	0	0	0	0	0	4	0	-4	0	-4	4	0	4	0	0	-4	-4	0	4	0	4	4	4	-4	4	4	0	
07	0	0	4	0	0	0	0	-4	4	-4	0	-4	0	0	0	4	4	4	0	4	0	0	0	4	4	4	0	-4	4	-4	-4	0	
08	0	0	0	0	-4	4	4	4	0	-4	-4	0	-4	0	0	4	0	-4	0	-4	-4	0	4	0	-4	4	0	0	0	0	-4	-4	
09	0	0	-4	-4	0	4	-4	0	4	0	0	-4	0	0	4	4	4	-4	0	0	0	-4	-4	0	0	-4	-4	0	0	0	4	-4	
0A	0	4	0	4	0	-4	-4	0	0	-4	0	4	-4	0	0	4	4	-4	4	4	4	4	0	0	0	0	0	0	-4	4	0	0	
0B	0	4	-4	0	-4	4	4	-4	4	0	4	0	0	0	-4	-4	0	-4	4	0	-4	4	0	0	0	4	0	4	0	4	0	0	
0C	0	-4	0	0	4	4	-4	0	0	4	4	4	0	0	4	0	4	0	4	-4	0	0	0	-4	0	4	4	-4	0	0	-4	0	
0D	0	4	4	4	0	4	-4	4	-4	0	0	0	4	0	0	0	0	0	4	0	-4	-4	0	4	4	-4	0	4	0	0	-4	0	
0E	0	0	0	-4	0	-4	-4	4	0	-4	0	0	0	0	-4	0	0	0	0	-4	0	-4	4	-4	4	0	-4	-4	4	4	0	4	
0F	0	0	-4	0	-4	4	-4	0	-4	0	4	-4	-4	0	0	0	-4	0	0	0	4	0	4	4	0	0	0	-4	-4	-4	0	4	
10	0	0	0	0	0	0	4	-4	-4	-4	4	4	-4	4	0	0	0	4	4	0	0	-4	0	-4	0	-4	0	-4	0	-4	0	-4	
11	0	0	-4	4	4	0	-4	0	0	0	0	0	0	4	-4	0	-4	-4	-4	4	-4	0	0	-4	4	4	0	0	0	-4	0	-4	
12	0	-4	0	4	4	0	4	0	-4	4	0	0	-4	-4	0	0	4	-4	0	0	0	0	4	4	4	0	-4	0	4	0	4	0	
13	0	4	-4	0	0	0	-4	-4	0	0	-4	4	0	4	4	0	4	0	4	0	-4	-4	4	4	4	0	0	0	0	4	0	4	0
14	0	4	0	0	0	0	4	0	-4	4	-4	0	0	4	4	4	-4	0	0	0	4	-4	-4	0	4	4	0	-4	0	4	0	0	
15	0	4	-4	-4	4	0	4	4	0	0	0	4	4	-4	0	4	0	0	0	4	0	0	4	0	0	-4	4	-4	0	-4	0	0	
16	0	0	0	-4	-4	0	-4	-4	-4	4	0	4	0	-4	-4	4	0	0	-4	0	4	0	0	0	0	0	0	4	4	0	-4	-4	
17	0	0	4	0	0	0	-4	0	0	0	-4	0	-4	-4	0	-4	-4	0	4	4	0	-4	0	0	-4	0	4	-4	4	0	4	-4	
18	0	0	0	0	4	4	0	0	4	0	0	4	0	4	0	-4	0	0	-4	4	4	-4	4	4	-4	0	-4	0	0	4	-4	0	
19	0	0	4	4	0	4	0	-4	0	-4	4	0	4	-4	4	4	-4	0	-4	0	0	0	4	-4	0	0	0	0	0	4	4	0	
1A	0	4	0	-4	0	4	0	-4	-4	0	-4	0	0	-4	0	-4	4	0	0	4	-4	0	0	-4	0	4	-4	0	-4	0	0	4	
1B	0	-4	-4	0	-4	-4	0	0	-4	0	4	4	-4	4	-4	0	0	0	0	0	-4	0	4	4	4	0	0	-4	0	0	0	-4	
1C	0	4	0	0	4	-4	0	-4	4	0	0	0	-4	-4	4	0	-4	-4	0	-4	0	-4	0	0	0	0	0	4	0	-4	-4	4	
1D	0	4	-4	4	0	-4	0	0	0	4	4	-4	0	-4	0	0	0	4	0	0	-4	0	0	0	-4	0	-4	-4	0	4	-4	-4	
1E	0	0	0	-4	0	-4	0	0	-4	0	4	-4	4	4	4	0	0	-4	4	4	0	0	4	0	-4	4	0	4	4	0	0	0	
1F	0	0	-4	0	4	4	0	4	0	-4	0	0	0	-4	0	0	-4	4	4	0	4	4	-4	0	0	4	-4	4	4	0	0	0	

4.4.2 Security against Differential Attacks

We evaluated the security of the f -960 and f -1920 permutations based on the principles of differential cryptanalysis. The results are as follows:

For the f -960 variant, it can be proven that the optimal two-round differential trail involves at least 6 active S-boxes, while the optimal three-round trail involves at least 17 active S-boxes. Since the maximum differential probability of each S-box is 2^{-4} , the probability of a 12-round differential trail is bounded by $2^{-4 \times 17 \times 4} = 2^{-272}$. Consequently, the time complexity for finding a collision for Duet-512 variant via differential trails exceeds the birthday bound.

For the f -1920 variant, the optimal two-round differential trail is proven to contain at least 8 active S-boxes, and the optimal three-round trail contains no fewer than 22 active S-boxes. With the maximum differential probability of each S-box being 2^{-4} , the probability of an 18-round differential trail is bounded by $2^{-4 \times 22 \times 6} = 2^{-528}$. Thus, the time complexity for finding a collision for Duet-1024 variant via differential trails also surpasses the birthday bound.

4.5 Statistical Randomness Test Results

We evaluate the statistical randomness of our algorithm using the NIST SP 800-22 statistical test suite. Following the recommended practice, we generate 1,000 sequences of length 10^6 bits for each of the three Duet variants (512, 768, and 1024). For each test, a sequence is considered to have passed if its P-value is at least 0.01, and the test is considered passed if the pass rate across all sequences is at least 98%.

Table 10 summarises the test results. Pass rates are given as X/Y, where X is the number of passing sequences and Y is the total number evaluated for that test. For most tests, Y =

1000; for RandomExcursions and RandomExcursionsVariant, Y is the number of sequences that exhibit at least one zero-crossing in the cumulative sum random walk—sequences that never return to zero are excluded, and the corresponding Y values are listed in the table. All 15 tests are passed for every Duet variant, demonstrating that the outputs of the Duet hash function exhibit statistical randomness and meet the requirements of the NIST SP 800-22 suite.

Table 10: Summary of NIST SP 800-22 test results for Duet.

Statistical Test	Duet-512	Duet-768	Duet-1024
Frequency	992/1000	981/1000	991/1000
BlockFrequency	988/1000	988/1000	992/1000
CumulativeSums	990/1000	986/1000	991/1000
Runs	993/1000	990/1000	989/1000
LongestRun	983/1000	993/1000	987/1000
Rank	994/1000	994/1000	992/1000
FFT	988/1000	988/1000	988/1000
NonOverlappingTemplate	980/1000	981/1000	981/1000
OverlappingTemplate	988/1000	986/1000	986/1000
Universal	993/1000	992/1000	981/1000
ApproximateEntropy	990/1000	987/1000	990/1000
RandomExcursions	585/599	629/637	628/638
RandomExcursionsVariant	590/599	628/637	627/638
Serial	995/1000	985/1000	985/1000
LinearComplexity	990/1000	990/1000	987/1000

5 Performance Evaluation

5.1 Performance analysis

Duet is implemented as a sponge construction with a serial dependency through the capacity state. For each padded message block, `absorb_block` first combines the message with state part A and applies `f1` together with the current capacity; it then permutes the same message block by `sigma`, combines it with state part B, and applies `f2` using the intermediate capacity. The capacity value produced by this pair becomes part of the next block state, so a single long message is dominated by the absorb loop and its two permutation calls per block.

The permutation is the main arithmetic component of this loop. Duet-512 uses a 960-bit state, while Duet-768 and Duet-1024 use 1920-bit states. Each round applies independent 5-word S-box columns, an 115/130 xor linear layer, round-constant injection, and an 164 layer composed of fixed 64-bit rotations and xors. This produces a regular instruction mix with little data-dependent control flow. Short messages include a larger fraction of interface, padding, final-block, and rate/capacity copy overhead, while larger S1–S8 inputs amortize these fixed costs and more directly reflect the steady-state permutation throughput. The SIMD implementations therefore concentrate on mapping the column and rotate/xor structure to NEON, AVX2, and AVX-512 registers while keeping the block-level capacity dependency unchanged.

5.2 Software Implementation

5.2.1 Benchmark Setup

This subsection reports the software implementation and self-evaluation results of Duet-512, Duet-768, and Duet-1024. The software packages follow the same submission-oriented organi-

zation as the corresponding implementation archive: a portable reference implementation, an x86 AVX2 optimized implementation, an ARMv8 NEON additional implementation, and an x86 AVX-512 additional implementation. All variants expose the same NGCC hash interface,

```
int CryptHash(int digest_len_bits, const unsigned char *msg,
              unsigned long long msg_len_bits, unsigned char *digest);
```

The benchmark uses the NGCC S1–S8 input sizes for hash algorithms, namely 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 bytes. Each case was executed with at least 100 iterations and was extended to at least one second of measurement time. The input messages were generated by a fixed LCG with seed `0x4e474343`. KAT generation used the bundled DRNG and KAT harness, and each generated KAT file was compared against the submitted `Test_Vectors/` file with the same name.

The evaluation environment and build configuration are summarized in Table 11. The average-cycle column in the performance tables is computed from the measured average time and the platform frequency: 2.0 GHz for the ARMv8 host and 3709.3569 MHz for the AMD EPYC server. The benchmark program also recorded raw architecture counter values (`cntvct_el0_ticks` on ARM and `rdtsc_ticks` on x86) during the measurement runs.

Table 11: Software evaluation environment and build configuration.

Platform	Processor and memory	OS / kernel	Compiler / build tool	Main compile flags
ARMv8 NEON	HiSilicon 920 2 vCPU single core: CPU 0	Ubuntu 24.04 Linux 6.8.0 aarch64	GCC 13.3.0 direct GCC build	-O3 -std=c99 -Wpedantic -Wall -Wextra
AMD EPYC Server AVX2	AMD EPYC 9654 384 CPU, 2 socket single core: CPU 0	Ubuntu 22.04 Linux 6.8.0 x86_64	GCC 11.4.0 direct GCC build	-O3 -std=c99 -mavx2 -mbmi -mbmi2 -madx -mtune=native
AMD EPYC Server AVX-512	AMD EPYC 9654 384 CPU, 2 socket single core: CPU 0	Ubuntu 22.04 Linux 6.8.0 x86_64	GCC 11.4.0 direct GCC build	-O3 -std=c99 -mavx2 -mavx512f -mavx512vl -mavx512bw -mavx512dq -mtune=native

5.2.2 Performance Results and Resource Consumption

The NGCC S1–S8 software performance results for the ARMv8 NEON implementation, the x86 AVX2 C implementation, and the x86 AVX-512 C implementation are reported in Tables 12, 13, and 14, respectively. The corresponding resource-consumption results are summarized in Table 15.

Table 12: ARMv8 NEON implementation, NGCC S1–S8 performance results.

Version	Level	Bytes	Time (ns)	Cycles	MB/s
Duet-512	S1	32	524.38	1048.76	61.02
	S2	128	1543.15	3086.30	82.95
	S3	512	5624.00	11248.00	91.04
	S4	1024	11231.11	22462.22	91.18
	S5	4096	43884.70	87769.40	93.34
	S6	8192	87206.39	174412.78	93.94
	S7	16384	174430.28	348860.56	93.93
	S8	65536	697420.47	1394840.94	93.97
Duet-768	S1	32	1497.14	2994.28	21.37
	S2	128	1491.93	2983.86	85.80
	S3	512	5935.81	11871.62	86.26
	S4	1024	11829.20	23658.40	86.57
	S5	4096	45742.54	91485.08	89.54
	S6	8192	89974.94	179949.88	91.05
	S7	16384	178446.90	356893.80	91.81
	S8	65536	712068.78	1424137.56	92.04
Duet-1024	S1	32	1484.47	2968.94	21.56
	S2	128	2946.24	5892.48	43.45
	S3	512	8792.77	17585.54	58.23
	S4	1024	16078.16	32156.32	63.69
	S5	4096	62792.87	125585.74	65.23
	S6	8192	125484.76	250969.52	65.28
	S7	16384	252611.01	505222.02	64.86
	S8	65536	996604.66	1993209.32	65.76

Table 13: x86 AVX2 C implementation, NGCC S1–S8 performance results.

Version	Level	Bytes	Time (ns)	Cycles	MB/s
Duet-512	S1	32	345.95	1283.25	92.50
	S2	128	1017.80	3775.38	125.76
	S3	512	3707.53	13752.55	138.10
	S4	1024	7398.25	27442.75	138.41
	S5	4096	28841.73	106984.27	142.02
	S6	8192	57332.77	212667.71	142.89
	S7	16384	114607.73	425120.97	142.96
	S8	65536	457625.88	1697497.72	143.21
Duet-768	S1	32	1421.68	5273.52	22.51
	S2	128	1418.73	5262.58	90.22
	S3	512	5632.38	20892.51	90.90
	S4	1024	11221.08	41622.99	91.26
	S5	4096	43417.34	161050.41	94.34
	S6	8192	85486.11	317098.49	95.83
	S7	16384	169455.41	628570.59	96.69
	S8	65536	674990.20	2503779.56	97.09
Duet-1024	S1	32	1414.53	5247.00	22.62
	S2	128	2809.74	10422.33	45.56
	S3	512	8395.01	31140.09	60.99
	S4	1024	15362.62	56985.44	66.66
	S5	4096	60049.04	222743.32	68.21
	S6	8192	120132.39	445613.91	68.19
	S7	16384	238703.98	885438.26	68.64
	S8	65536	953151.98	3535580.87	68.76

Table 14: x86 AVX-512 C implementation, NGCC S1–S8 performance results.

Version	Level	Bytes	Time (ns)	Cycles	MB/s
Duet-512	S1	32	316.72	1174.83	101.03
	S2	128	934.11	3464.95	137.03
	S3	512	3396.92	12600.39	150.72
	S4	1024	6777.19	25139.02	151.10
	S5	4096	26433.45	98051.10	154.96
	S6	8192	52519.69	194814.27	155.98
	S7	16384	104992.15	389453.36	156.05
	S8	65536	419394.31	1555683.18	156.26
Duet-768	S1	32	1051.79	3901.46	30.42
	S2	128	1047.90	3887.04	122.15
	S3	512	4104.30	15224.31	124.75
	S4	1024	8184.39	30358.82	125.12
	S5	4096	31555.03	117048.87	129.80
	S6	8192	62068.66	230234.81	131.98
	S7	16384	123078.29	456541.30	133.12
	S8	65536	490286.29	1818646.83	133.67
Duet-1024	S1	32	1037.93	3850.05	30.83
	S2	128	2043.11	7578.62	62.65
	S3	512	6065.14	22497.77	84.42
	S4	1024	11094.85	41154.76	92.30
	S5	4096	43265.47	160487.07	94.67
	S6	8192	86622.99	321315.59	94.57
	S7	16384	172306.94	639147.94	95.09
	S8	65536	689194.95	2556470.04	95.09

The measurements show the expected transition from fixed-cost short-message behavior to steady-state sponge absorption. S1 and S2 include a comparatively large fraction of call setup, padding, and finalization overhead, while the larger inputs amortize those costs across many absorbed blocks. On the AMD EPYC platform, the AVX-512 implementation improves the long-message throughput for all three instances in the S8 test, with the largest gains for Duet-768 and Duet-1024. The ARMv8 NEON results are lower in absolute throughput on the tested cloud instance, but they scale smoothly from S3 to S8 and confirm that the portable submission package works correctly on an aarch64 target.

Table 15: Software resource-consumption results. Static total is `text+data+bss`; peak RSS is the maximum resident set size observed during S1–S8 execution.

Platform	Version	Text	Data	BSS	Static Total	Peak RSS
ARMv8 NEON	Duet-512	12130	928	8	13066	1441792
	Duet-768	16075	936	8	17019	1441792
	Duet-1024	14307	936	8	15251	1441792
x86 AVX2	Duet-512	14019	856	16	14891	1912832
	Duet-768	20964	872	16	21852	1789952
	Duet-1024	19581	872	16	20469	1916928
x86 AVX-512	Duet-512	13394	856	16	14266	1921024
	Duet-768	17972	872	16	18860	1785856
	Duet-1024	16285	872	16	17173	1904640

The static binary sizes remain modest across all optimized implementations. The x86 AVX2 implementation has the largest static footprint for Duet-768 and Duet-1024 because its vectorized C path expands the 1920-bit permutation logic, while the AVX-512 path represents the six-column state more compactly in wide registers. Peak RSS is close across all benchmarks and is dominated by the benchmark harness and message buffer rather than by the hash state itself.

5.3 Hardware Implementation

This subsection presents the Verilog hardware simulation implementation results of the Duet-512, Duet-768 and Duet-1024 algorithms. Synthesis was performed in Synopsys Design Compiler L-2016.03-SP1, targeting the TSMC 65nm process library `tcbn65gplusbwp12tbc_ccs`. The Duet-512 implementation was synthesised under a clock frequency of 1.33 GHz. The Duet-768 and Duet-1024 implementation was synthesised under a clock frequency of 1 GHz.

The implementation results are shown in Table 16.

Table 16: Hardware implementation performance results

Version	Message Length (Kb)	Clock Cycles (cycles)	Throughput (Mb/s)	Latency (ns)	Synthesized Area (μm^2)	Normalized Area (GE)	Power (mW)
Duet-512	1	88	15152	66	149012.16	77610.50	9.1728
	8	639	17093	479.25			
	256	19808	17646	14856			
	8192	633535	17655	475151.25			
Duet-768	1	42	24381	42	387607.21	201878.75	20.9915
	8	329	24900	329			
	256	9882	26527	9882			
	8192	316152	26546	316152			
Duet-1024	1	83	12337	83	381518.41	198707.50	20.3269
	8	452	18124	452			
	256	14023	18694	14023			
	8192	447884	18731	447884			

6 Features

Duet combines a serial two-stage update with a shared capacity branch, capacity feed-forward, and capacity-based squeezing. During absorption, each message block updates the two rate branches sequentially through f_1 and f_2 . During squeezing, output is extracted from the capacity branch in h -bit fragments. This uniform structure supports both the fixed-length Duet- h digest and an extendable-output interface.

6.1 Innovative Aspects of the Algorithm

- **An alternating structure with beyond-birthday bound indifferentiability.** Duet adopts the alternating two-branch schedule from our previous Alternating Sponge framework [12], for which indifferentiability in the ideal-permutation model was proven up to the generic threshold $2^{\min\{\frac{2c}{3}, \frac{r+c}{2}\}}$ that is beyond the birthday bound $2^{c/2}$. Since Duet further introduces capacity feed-forward and capacity-based output, this result is used as structural security background and design rationale.
- **A serially coupled dual-branch structure.** Duet maintains two rate branches connected by one shared capacity branch. For each message block, the first rate branch is updated through f_1 , and the resulting capacity value is immediately used in the second-stage call to f_2 for updating the second rate branch. This creates a fixed serial dependency between the two branches: each block affects both rate branches, while the capacity branch carries the coupling information across the two stages. At the same time, both stages use the same $(r + c)$ -bit permutation interface, which keeps the state update regular and easy to specify.

- **A capacity feed-forward mechanism with capacity-based output.** After each permutation call, Duet updates the capacity branch by combining the input capacity value with the capacity part of the permutation output. During squeezing, fixed-length digests and variable-length outputs are extracted from the capacity branch rather than from the rate branches. This design connects the capacity feed-forward state evolution with the output interface, and its effect is explicitly reflected in the preimage and second-preimage analyses.
- **Permutations f with provably bounded differential probability.** By evaluating the security of reduced-round f functions, it can be proven that the differential probability of the 12-round f -960 is no higher than 2^{-256} , and that of the 18-round f -1920 is no higher than 2^{-512} . It means the time complexity for finding a collision for Duet via differential trails exceeds the birthday bound.

6.2 Simplicity and Structural Uniformity

The overall workflow of Duet consists of message padding, absorbing, and squeezing. Its structure remains close to the standard sponge paradigm, which makes the construction relatively easy to describe, implement, and analyze. Rather than introducing a complex external control flow, the algorithm updates its state through a fixed two-stage alternating permutation rule.

In terms of state representation, the algorithm is organized around two rate branches and a capacity branch. Both the absorbing phase and the squeezing phase reuse the same permutation-call interface: a rate part is paired with the current capacity value, the permutation is applied, and the capacity output is XORed with the capacity input. This uniform wrapper makes the absorbing and squeezing phases structurally consistent.

The main operations used by the algorithm are bitwise XOR, bit-string concatenation, the linear permutation σ , truncation of the capacity branch, and calls to the underlying permutation. This small set of operations helps keep the specification clear and makes it easier to instantiate the construction with different underlying permutations or parameter choices.

6.3 Flexibility and Parameter Scalability

The output interface of Duet supports any requested length $\nu \geq 1$ without changing the absorbing procedure. For the Duet- h instance, $\nu = h$ gives the fixed-length digest, while $\nu > h$ continues the alternating squeezing process to produce additional output fragments, providing an interface similar to SHAKE [2].

In addition to ordinary hashing and XOF output, Duet can also serve as the core construction for derived interfaces such as MAC and KDF. These interfaces can follow the domain-separation and input-framing principles used in SHA-3-derived functions, such as cSHAKE, KMAC, and TupleHash [3], as well as the common KDF practice of encoding keying material, salt, context information, and output length [4]. Different applications can prepend fixed mode identifiers and use injective, self-delimiting encodings for their inputs. In this way, inputs from different application domains are mapped to distinct framed strings before being processed by the same Duet core, without changing the internal state-update rule.

These interfaces illustrate the application-level flexibility of Duet. The concrete security of MAC, KDF, or other derived modes still depends on the entropy of the secret input, the selected tag or output length, the capacity parameter of the chosen instance, and the exact framing method.

6.4 Implementation Advantages

- **Software-friendly implementation:** The underlying permutation mainly uses logical operations, cyclic shifts, and XORs, which can be efficiently mapped to word-level opera-

tions on 64-bit processors. The 5-bit S-box can also be implemented in a bit-sliced form, avoiding large lookup tables and data-dependent memory accesses.

- **High-throughput hardware structure:** The round function consists of simple S-box computations, cyclic shifts, and XOR networks, which can be implemented with a regular and highly parallel datapath. Together with the relatively small number of rounds, this structure reduces the number of clock cycles required for each permutation call and supports high throughput in practical hardware implementations.
- **Low resource requirement:** Since f_1 and f_2 are invoked in sequence, a compact implementation can reuse one permutation datapath for the two stages. The state only contains two rate branches and one shared capacity branch, so the capacity feed-forward operation does not require duplicating a full $(r + c)$ -bit permutation state.

References

- [1] Guido Bertoni, Joan Daemen, Michael Peeters, and Gilles Van Assche. Cryptographic sponge functions. 2011. Available at <https://keccak.team/files/CSF-0.1.pdf>.
- [2] National Institute of Standards and Technology. SHA-3 standard: permutation-based hash and extendable-output functions. *Federal Information Processing Standards Publication 202*, 2015. doi: <https://doi.org/10.6028/NIST.FIPS.202>. Available at <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>.
- [3] National Institute of Standards and Technology. SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash. *NIST Special Publication 800-185*, 2016. doi: <https://doi.org/10.6028/NIST.SP.800-185>. Available at <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf>.
- [4] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). *RFC 5869*, 2010. doi: <https://doi.org/10.17487/RFC5869>. Available at <https://www.rfc-editor.org/rfc/rfc5869>.
- [5] Arghya Bhattacharjee, Avik Chakraborti, Nilanjan Datta, Cuauhtemoc Mancillas-López, and Mridul Nandi. ISAP+: ISAP with fast authentication. *Cryptology ePrint Archive, Paper 2022/1591*, 2022. Available at <https://eprint.iacr.org/2022/1591.pdf>.
- [6] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-based hash from non-idealized assumptions: adding feed-forward to sponge. *Cryptology ePrint Archive, Paper 2025/1006*, 2025. Available at <https://eprint.iacr.org/2025/1006.pdf>.
- [7] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. *Cryptology ePrint Archive, Paper 2025/963*, 2025. Available at <https://eprint.iacr.org/2025/963.pdf>.
- [8] John Kelsey and Bruce Schneier. Second preimages on n -bit hash functions for much less than 2^n work. *Cryptology ePrint Archive, Paper 2004/304*, 2004. Available at <https://eprint.iacr.org/2004/304.pdf>.
- [9] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC 1996)*, pages 212–219, 1996. doi: <https://doi.org/10.1145/237814.237866>. Available at <https://arxiv.org/abs/quant-ph/9605043>.
- [10] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN'98: Theoretical Informatics*, Lecture Notes in Computer Science, volume 1380, pages 163–169. Springer, 1998.
- [11] Yaoyun Shi. Quantum lower bounds for the collision and the element distinctness problems. In *Proceedings of the 43rd IEEE Symposium on Foundations of Computer Science (FOCS 2002)*, pages 513–519, 2002. Available at <https://arxiv.org/abs/quant-ph/0112086>.
- [12] Ziyang Luo, Yaobin Shen, Hailun Yan, Lei Wang, and Dawu Gu. Alternating Sponge: A Low-Memory Hash Function with Beyond-Birthday-Bound Security. *Cryptology ePrint Archive, Paper 2026/707*, 2026. Available at <https://eprint.iacr.org/2026/707>.

- [13] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption – FSE 2004*, Lecture Notes in Computer Science, volume 3017, pages 371–388. Springer, 2004. doi: https://doi.org/10.1007/978-3-540-25937-4_24. Available at <https://www.iacr.org/archive/fse2004/30170373/30170373.pdf>.
- [14] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indistinguishability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, 2008. Available at <https://keccak.team/files/SpongeIndistinguishability.pdf>.
- [15] Ueli Maurer, Renato Renner, and Clemens Holenstein. Indistinguishability, impossibility results on reductions, and applications to the random oracle methodology. In *Theory of Cryptography – TCC 2004*, Lecture Notes in Computer Science, volume 2951, pages 21–39. Springer, 2004. doi: https://doi.org/10.1007/978-3-540-24638-1_2. Available at <https://eprint.iacr.org/2003/161.pdf>.